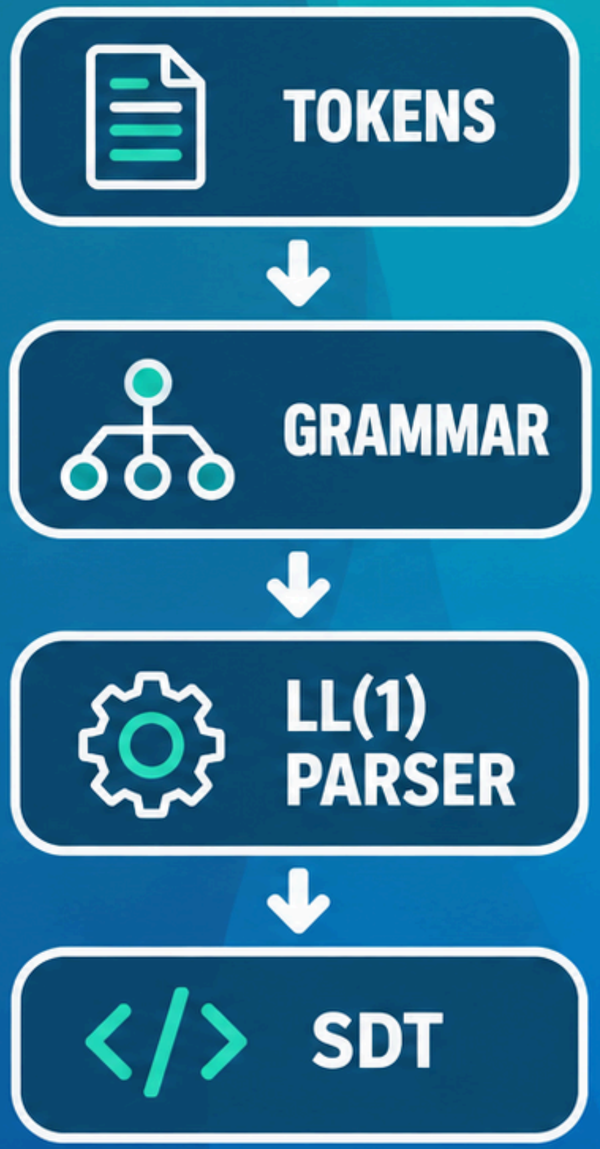
UNAM · Compilers · Team 5

PARSER DESIGN PROCESS

COMPILER PROJECT · PARSER & SDT



PROJECT GOAL

Build a syntax and semantic analyzer that checks the structure and meaning of a small program.

EXAMPLE INPUT

```
int x = 10;
```

EXPECTED RESULT

Parsing Success!

SDT Verified!

01 LEXER

Produces tokens

`int id = num ;`

02 LL(1) PARSER

Checks grammar

Builds the parse tree

03 SDT

Checks semantic rules

Types and declarations

OUR WORKFLOW

We developed the analyzer step by step, starting with the tokens from our previous Lexer.

FROM SOURCE CODE

↓

TO VALIDATED OUTPUT

01 Adapt the grammar
Extend declarations into statements and expressions

↓

02 Check LL(1) conditions
No left recursion; unambiguous alternatives

↓

03 Calculate FIRST and FOLLOW
Find valid starting and following terminals

↓

04 Build the parsing table
Choose one production for each table cell

↓

05 Run the parser
Use tokens, a stack, and a predictive table

↓

06 Evaluate SDT and test
Check types, symbols, and errors

GRAMMAR DESIGN

We extended our Lexer grammar to support several statements and arithmetic expressions.

27 PRODUCTIONS

12 NON-TERMINALS

16 TERMINALS

WHAT CHANGED?

- Multiple declarations and assignments
- Arithmetic: + − × / %
- Parentheses, numbers, and identifiers

KEY PRODUCTIONS · FROM OUR REPORT

```
Program   → Stmt StmtList
StmtList  → Stmt StmtList | ε
Stmt      → Decl | Assign
Decl      → Type id Decl′
Decl′     → ; | = Expr ;
Assign    → id = Expr ;
Expr      → Term Expr′
Term      → Factor Term′
```

WHY TOP-DOWN LL(1)?

Our grammar can be parsed predictively, from left to right, using only one lookahead token.

- TOP-DOWN tree construction
- NO backtracking
- ONE lookahead token

FIRST + FOLLOW → PREDICTIVE TABLE

FIRST identifies possible starting tokens. FOLLOW helps decide when to use ε.

NON-TERMINAL	LOOKAHEAD	PRODUCTION
`Stmt`	`int`	Stmt → Decl
`Stmt`	`id`	Stmt → Assign
`StmtList`	`$`	StmtList → ε

12 × 17 TABLE • 51 FILLED CELLS • NO CONFLICTS

The parser uses the table + a stack to select each rule.

PARSER + SDT

The parser constructs a parse tree. After parsing succeeds, SDT evaluates semantic attributes on that tree.

PARSER · SYNTAX

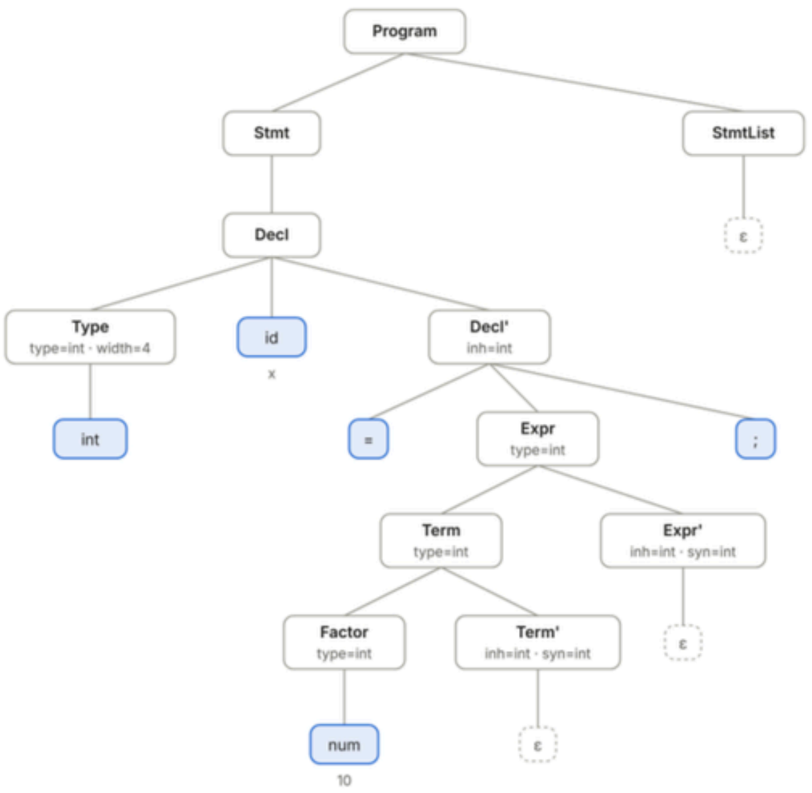
Grammar rules + stack + table

SDT · SEMANTICS

Types + symbol table + attributes

CHECKS

- Declare variables before use
- Reject incompatible assignments
- Record types, sizes, and addresses



ANNOTATED TREE · int x = 10;

TESTING & RESULTS

We tested three cases to distinguish syntax errors from semantic errors.

VALID PROGRAM

```
int x = 10;
```

Parsing Success!

SDT Verified!

SYNTAX ERROR

```
int x 10;
```

Parsing Error

Expected “=” or “;”

SEMANTIC ERROR

```
int x = 10;    float y = x * 2.5;
x = y;
```

Parsing Success!

SDT Error — line 3

SYNTAX = STRUCTURE · SEMANTICS = MEANING